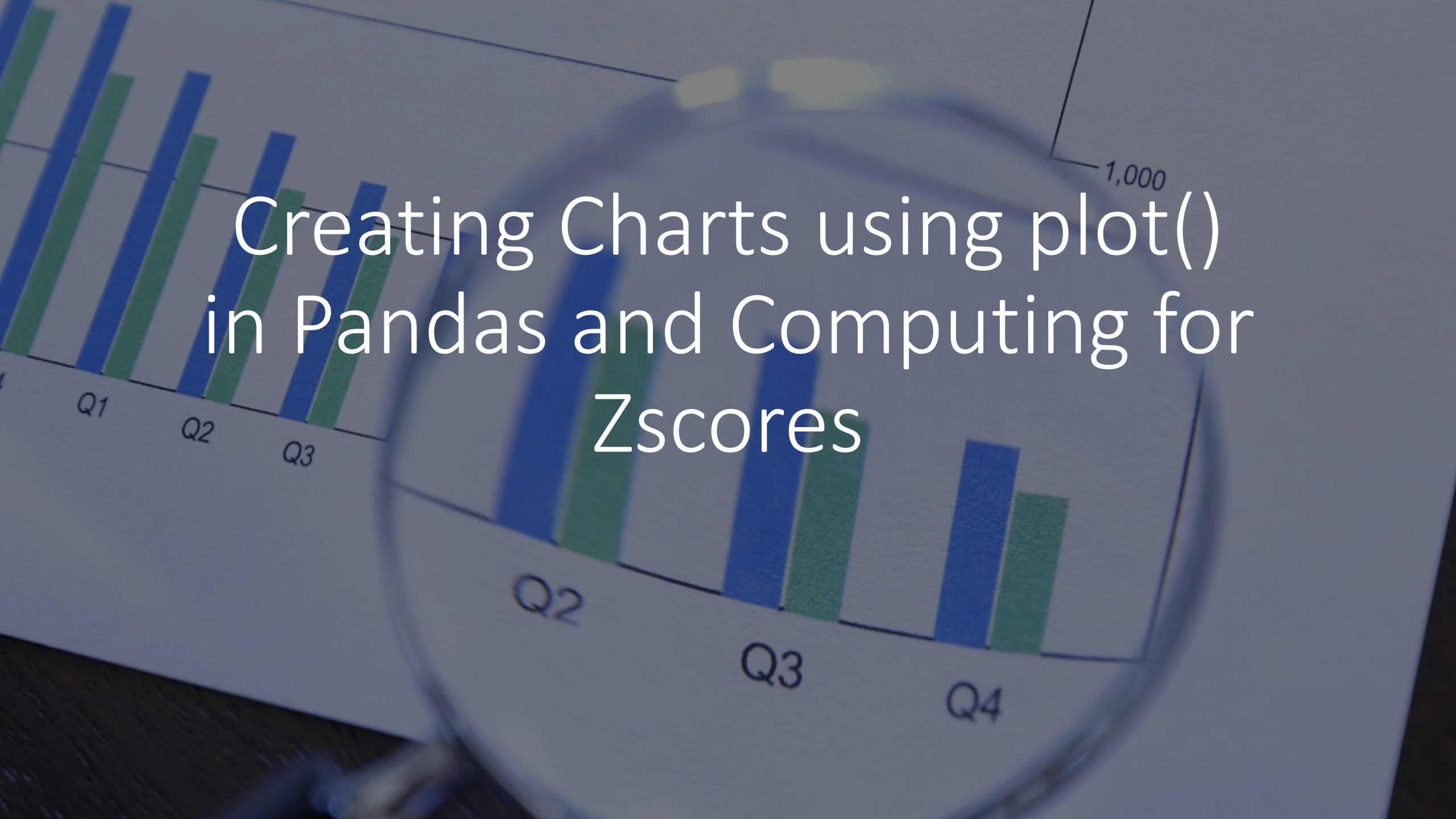




Creating Charts using plot() in Pandas and Computing for Zscores

Objectives



Discuss the purpose of `plot()` method



Demonstrate the use of `plot()` method in creating diagrams such as boxplots, histograms, etc.



Demonstrate the use of `plot()` method in creating diagrams such as boxplots, histograms, etc.

How to Upload a .CSV file in Google Colab

- Use this code to upload .CSV file data set in Colab:

```
import pandas as pd

from google.colab import files
uploaded=files.upload()
```

What is a plot() method?

- Pandas uses the plot() method to create diagrams.

Kinds of plot produced:

- The kind of plot to produce:
- 'line' : line plot (default)
- 'bar' : vertical bar plot
- 'barh' : horizontal bar plot
- 'hist' : histogram
- 'box' : boxplot
- 'kde' : Kernel Density Estimation plot
- 'density' : same as 'kde'
- 'area' : area plot
- 'pie' : pie plot
- 'scatter' : scatter plot (DataFrame only)
- 'hexbin' : hexbin plot (DataFrame only)

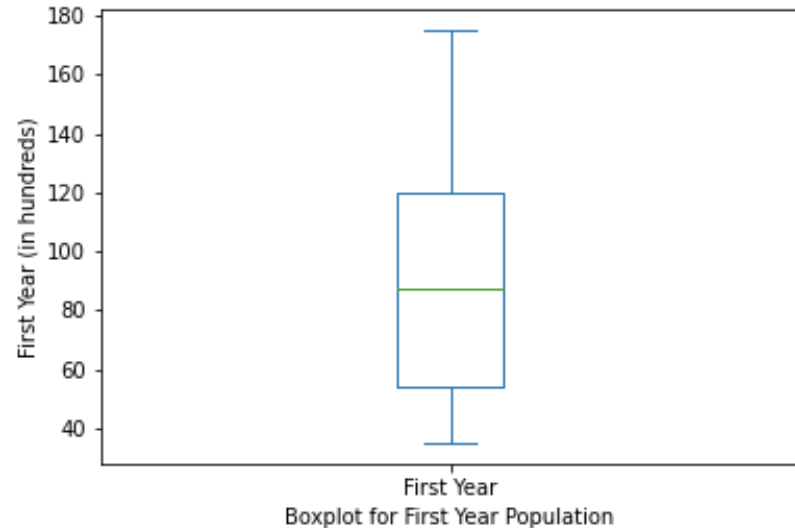
What is a boxplot?

- **box-and-whisker plot**, also called **boxplot** or **box plot**, graph that summarizes numerical data based on quartiles, which divide a data set into fourths.
- The box-and-whisker plot is useful for revealing the central tendency and variability of a data set, the distribution of the data.

Creating box and whisker plot

```
#boxplot
ax=(studdf[['First Year']]).plot.box()
ax.set_ylabel('First Year (in hundreds)')
ax.set_xlabel('Boxplot for First Year Population')
```

Text(0.5, 0, 'Boxplot for First Year Population')



```
studdf=pd.DataFrame(studdata)
print (studdf)
```

	Course	First Year	Second Year	Third Year	Fourth Year
0	BSCS	35	20	18	18
1	BSIT	90	75	70	68
2	BSCOE	85	64	60	56
3	BSECE	54	48	40	35
4	BSEE	54	60	58	50
5	BSA	110	85	80	75
6	BSBA	150	120	110	105
7	BSCRIM	175	130	110	98

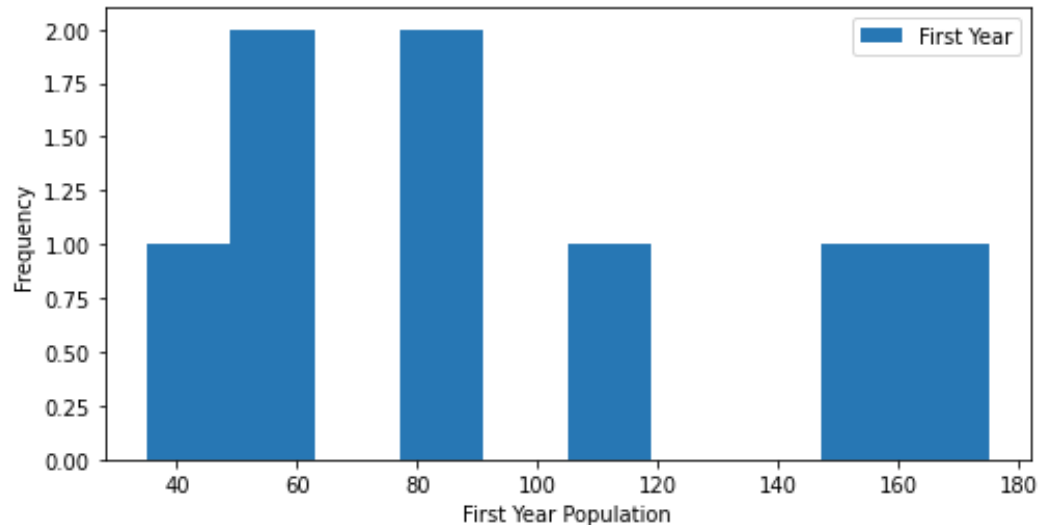
What is a histogram?

- It is similar to a bar chart, but a histogram groups numbers into **ranges** .
- The height of each bar shows how many fall into each range.

Creating a histogram

```
#creating histograms  
ax=(studdf[['First Year']]).plot.hist(figsize=(8,4))  
ax.set_xlabel('First Year Population')
```

```
Text(0.5, 0, 'First Year Population')
```



```
studdf=pd.DataFrame(studdata)  
print (studdf)
```

	Course	First Year	Second Year	Third Year	Fourth Year
0	BSCS	35	20	18	18
1	BSIT	90	75	70	68
2	BSCOE	85	64	60	56
3	BSECE	54	48	40	35
4	BSEE	54	60	58	50
5	BSA	110	85	80	75
6	BSBA	150	120	110	105
7	BSCRIM	175	130	110	98

What is a bar chart?

- A **Bar Graph** (also called Bar Chart) is a graphical display of data using bars of different heights.
- A **bar chart** is a graph with rectangular bars. The graph usually compares different categories.
- A **bar chart** is a way of summarizing a set of categorical data.

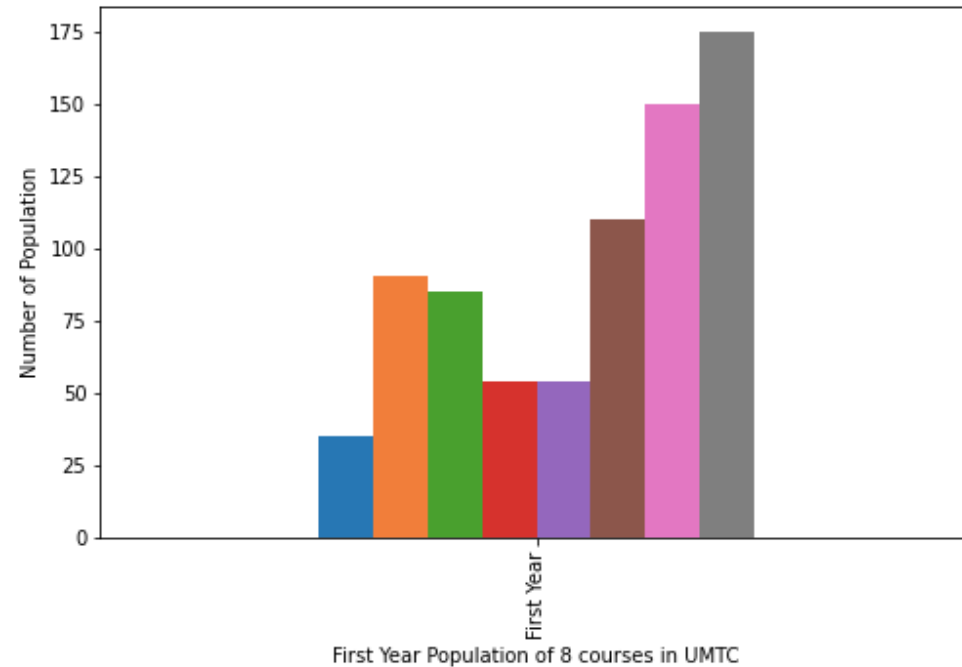
Creating a bar chart

```
studdf=pd.DataFrame(studdata)
print (studdf)
```

	Course	First Year	Second Year	Third Year	Fourth Year
0	BSCS	35	20	18	18
1	BSIT	90	75	70	68
2	BSCOE	85	64	60	56
3	BSECE	54	48	40	35
4	BSEE	54	60	58	50
5	BSA	110	85	80	75
6	BSBA	150	120	110	105
7	BSCRIM	175	130	110	98

```
#creating bar chart
ax=studdf[['First Year',]].transpose().plot.bar(figsize=(8,5), legend=False)
ax.set_xlabel('First Year Population of 8 courses in UMTC')
ax.set_ylabel('Number of Population')
```

Text(0, 0.5, 'Number of Population')

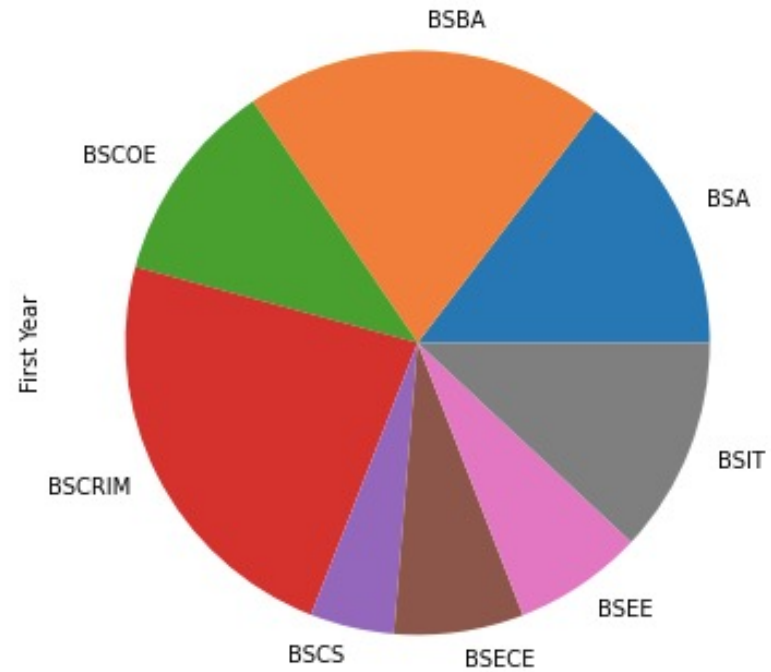


What is a pie chart?

- A Pie Chart is a type of graph that displays data in a circular graph. The pieces of the graph are proportional to the fraction of the whole in each category.
- In other words, **each slice of the pie is relative to the size of that category** in the group as a whole.

Creating Pie chart

```
#creating pie chart  
ax=studdf.groupby(['Course']).sum().plot(kind='pie', y='First Year', legend=False, figsize=(6,8))
```



Coding for Quartiles and Percentiles & IQR

```
[ ] #getting the quantile (percentile)
    quan75stud1=studdf[['First Year']].quantile(0.75)
    print(quan75stud1)
```

```
First Year    120.0
Name: 0.75, dtype: float64
```

```
[ ] quan50stud1=studdf[['First Year']].quantile(0.50)
    print (quan50stud1)
```

```
First Year    87.5
Name: 0.5, dtype: float64
```

```
[ ] #getting the quantile (percentile)
    quan25stud1=studdf[['First Year']].quantile(0.25)
    print(quan25stud1)
```

```
First Year    54.0
Name: 0.25, dtype: float64
```

```
[ ] #getting the IQR
    IQRstud1=studdf[['First Year']].quantile(0.75)-studdf[['First Year']].quantile(0.25)
    print (IQRstud1)
```

```
First Year    66.0
dtype: float64
```

Computing for Zscores in Colab

- In order to calculate the z-score, we need to first calculate the mean and the standard deviation of an array.
- To calculate the standard deviation from scratch:

```
# Calculate the Standard Deviation in Python
mean = sum(values) / len(values)
differences = [(value - mean)**2 for value in values]
sum_of_differences = sum(differences)
standard_deviation = (sum_of_differences / (len(values) - 1)) **
0.5

print(standard_deviation)
# Returns: 1.3443074553223537
```

How to Use Scipy to Calculate a Z-Score

The most common way to calculate z-scores in Python is to use the `scipy` module.

The `zscore()` function takes an array of values and returns an array containing their z-scores.

It implicitly handles calculating the mean and the standard deviation, so we don't need to calculate those ourselves.

How to Use Scipy to Calculate a Z-Score

```
# Calculate the z-score from with scipy
import scipy.stats as stats
values = [4,5,6,6,6,7,8,12,13,13,14,18]

zscores = stats.zscore(values)
print(zscores)

# Returns: [-1.2493901  -1.01512945 -0.78086881 -0.78086881
-0.78086881 -0.54660817 -0.31234752  0.62469505  0.85895569
0.85895569  1.09321633  2.0302589 ]
```

Data Frame with random values

```
import pandas as pd

data = pd.DataFrame(np.random.randint(0, 10, size=(5, 3)), columns=['A', 'B', 'C'])
data
```



	A	B	C
0	4	4	2
1	5	8	5
2	2	9	3
3	0	8	8
4	3	5	0



```
data.apply(stats.zscore)
```



	A	B	C
0	0.697486	-1.443990	-0.586588
1	1.278724	0.618853	0.513265
2	-0.464991	1.134563	-0.219971
3	-1.627467	0.618853	1.613118
4	0.116248	-0.928279	-1.319824

Sample code for Z-score

```
▶ import numpy as np
data = np.array([[5, 6, 7, 7, 8],
                 [8, 8, 8, 9, 9],
                 [2, 2, 4, 4, 5]])
stats.zscore(data, axis=1)

☐→ array([[ -1.56892908, -0.58834841,  0.39223227,  0.39223227,  1.37281295],
          [-0.81649658, -0.81649658, -0.81649658,  1.22474487,  1.22474487],
          [-1.16666667, -1.16666667,  0.5         ,  0.5         ,  1.33333333]])
```

```
[ ] import pandas as pd
import numpy as np

examdata={'Name': ['Student 1', 'Student 2','Student 3','Student 4', 'Student 5', 'Student 6', 'Student 7', 'Student 8','Student 9', 'Student 10', 'Student 11',
                  'Student 12', 'Student 13', 'Student 14', 'Student 15', 'Student 16', 'Student 17', 'Student 18', 'Student 19', 'Student 20', 'Student 21',
                  'Student 22', 'Student 23', 'Student 24', 'Student 25'],
          'Math 103/L Second Exam Scores':[48, 37, 63, 29, 30, 33, 38, 31, 35, 46,54, 33, 50, 40,45, 38,40, 49, 50,52, 28, 52, 47, 54, 58],

          }
```

```
▶ examdf=pd.DataFrame(examdata)
print(examdf)
```

```
↳
```

	Name	Math 103/L Second Exam Scores
0	Student 1	48
1	Student 2	37
2	Student 3	63
3	Student 4	29
4	Student 5	30
5	Student 6	33
6	Student 7	38
7	Student 8	31
8	Student 9	35
9	Student 10	46
10	Student 11	54
11	Student 12	33
12	Student 13	50
13	Student 14	40
14	Student 15	45
15	Student 16	38
16	Student 17	40
17	Student 18	49
18	Student 19	50
19	Student 20	52
20	Student 21	28
21	Student 22	52
22	Student 23	47
23	Student 24	54
24	Student 25	58



#computing for the zcores

```
examdf['Math 103/L 2nd Exam ZScores']=stats.zscore(examdf['Math 103/L Second Exam Scores'])  
print(examdf)
```



	Name	Math 103/L Second Exam Scores	Math 103/L 2nd Exam ZScores
0	Student 1	48	0.498488
1	Student 2	37	-0.643880
2	Student 3	63	2.056262
3	Student 4	29	-1.474693
4	Student 5	30	-1.370841
5	Student 6	33	-1.059287
6	Student 7	38	-0.540028
7	Student 8	31	-1.266990
8	Student 9	35	-0.851583
9	Student 10	46	0.290785
10	Student 11	54	1.121598
11	Student 12	33	-1.059287
12	Student 13	50	0.706191
13	Student 14	40	-0.332325
14	Student 15	45	0.186933
15	Student 16	38	-0.540028
16	Student 17	40	-0.332325
17	Student 18	49	0.602339
18	Student 19	50	0.706191
19	Student 20	52	0.913894
20	Student 21	28	-1.578545
21	Student 22	52	0.913894
22	Student 23	47	0.394636
23	Student 24	54	1.121598
24	Student 25	58	1.537004

Example 2

```
#computing for the zscores
import pandas as pd
import numpy as np
from scipy.stats import zscore

datal=np.array([[48, 37, 63, 29, 30, 33, 38, 31, 35, 46,54, 33, 50, 40,45, 38,40, 49, 50,52, 28, 52, 47, 54, 58]])
stats.zscore(datal, axis=1)
```



```
array([[ 0.49848779, -0.64388006,  2.05626214, -1.47469305, -1.37084142,
        -1.05928656, -0.54002844, -1.2669898 , -0.85158331,  0.29078454,
         1.12159753, -1.05928656,  0.70619104, -0.33232519,  0.18693292,
        -0.54002844, -0.33232519,  0.60233941,  0.70619104,  0.91389428,
        -1.57854467,  0.91389428,  0.39463617,  1.12159753,  1.53700402]])
```

References in Creating diagrams using plots in Pandas

- <https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.plot.html>